Label 2 ML Handoff

Recognizing InfiniBand network contention during distributed GPU training

Audience: teammates who will train the machine-learning model from the collected CSV files

What is provided	What the ML teammate does
Telemetry CSV files from two GPU nodes	Build a time-aligned training table
InfiniBand stress-marker CSV files	Create Label 2 ground truth
DDP and NCCL node logs	Train and evaluate a classifier
RDMA stress logs	Report which signals identify contention

Short answer: yes. The teammate can train a first model using the CSV files, provided the stress-marker files are included and the labels are created from their timestamps. The HPC scripts are useful for collecting more data, but they are not required for model training.

1. What Label 2 represents

Label 2 is a controlled distributed-training condition in which an additional RDMA workload creates traffic on the same InfiniBand fabric used by NCCL. The extra workload is generated with **ib_write_bw** between the two allocated GPU nodes. A telemetry poller records GPU and InfiniBand counters every approximately 0.5 seconds.

Each GPU computes a training step	Ranks exchange gradients	NCCL and RDMA traffic share the link	Counters are sampled and written to CSV

> This is a network-contention experiment, not a packet-delay injection experiment. It intentionally increases InfiniBand traffic and observes the effect on communication and training progress.

The supplied example run used job 105175 on g8 and g11. Both node logs confirm NCCL used NET/IB with mlx4_0:1, and both stress markers identify an ib_write_bw interval of about 123 seconds.

What the team is trying to learn

During Label 2, the model should recognize a pattern rather than depend on one threshold:

• InfiniBand transmit traffic and packet rates increase.

• The InfiniBand transmit-wait counter increases, indicating the port spent more time waiting to transmit.

• DDP step time may increase because NCCL communication takes longer or ranks wait for one another.

• GPU utilization can change, but GPU utilization alone is not a reliable Label 2 detector.

2. Files and their roles

File type	Purpose	Used by the model?
telemetry_*.csv	GPU and InfiniBand measurements over time	Yes - primary features
stress_active_*.csv	Start and end timestamps of ib_write_bw contention	Yes - create target label
node_*.log	NCCL path, DDP step times, job configuration	Optional - extract step time and validate run
rdma_stress_*.log	Confirms RDMA workload and reports bandwidth	No - experiment evidence and metadata

> The stress-marker CSV is essential. Without it, the teammate has measurements but no trustworthy ground-truth time interval for Label 2.

Ground-truth construction

For every telemetry row, compare its **timestamp** with the stress start and end timestamps. Assign **label = 2** when the timestamp falls inside the interval. Assign **label = 0** outside the interval. If Label 1 and other labels are added later, use their own experiment markers and assign the corresponding label.

```
# Conceptual label creation
telemetry["label"] = 0
telemetry.loc[
    (telemetry["timestamp"] >= stress_start) &
    (telemetry["timestamp"] <= stress_end),
    "label"
] = 2
```

3. Telemetry columns: what they mean

Column	Meaning	Recommended use
gpu_util_pct	GPU utilization percentage	Feature; use with network signals
gpu_mem_used_mb	GPU memory in use	Feature for workload context

Column	Meaning	Recommended use
ib_port_xmit_data_delta	Bytes transmitted during the sample interval	Feature; traffic intensity
ib_port_rcv_data_delta	Bytes received during the sample interval	Feature; traffic intensity
ib_port_xmit_packets_delta	Packets transmitted during the interval	Feature; packet rate
ib_port_rcv_packets_delta	Packets received during the interval	Feature; packet rate
ib_port_xmit_wait_delta	Increase in transmit-wait counter during the interval	Primary congestion feature
***_discards_delta**	Packets discarded during the interval	Error/congestion feature
***_rnr_delta**	Receiver-not-ready events	RDMA communication-health feature
***_to_delta**	Queue-pair timeout events	Failure-severity feature

> Use the interval delta columns rather than the cumulative counters whenever possible. Cumulative counters grow throughout a job and can cause a model to learn job age instead of contention.

Recommended first feature set

```
gpu_util_pct
gpu_mem_used_mb
ib_port_xmit_data_delta
ib_port_rcv_data_delta
ib_port_xmit_packets_delta
ib_port_rcv_packets_delta
ib_port_xmit_wait_delta
ib_port_xmit_discards_delta
ib_sq_num_rnr_delta
ib_sq_num_to_delta
ddp_step_time_ms  # add by parsing node_0_0.log
```

4. Adding DDP step time

The telemetry CSV does not currently contain DDP step time. The node log records lines such as **step_time_ms=21.183**. Extract the log timestamp from the **wall=...** field, then align each step-time observation to the nearest telemetry timestamp or aggregate it into the same 0.5-second window.

> A step time is a full distributed iteration measurement. It can include forward computation, backward computation, NCCL gradient synchronization, rank waiting, optimizer work, and the final CUDA synchronization. It is not pure GPU compute.

```
# Example log pattern to parse
step_time_ms=21.183 wall=1793847778.934761

# Suggested derived features
ddp_step_time_ms
ddp_step_time_rolling_mean
ddp_step_time_rolling_p95
```

If step-time extraction is not ready for the first model, train the first version using only the telemetry features. Add step time in the next data-preparation revision.

5. Data preparation workflow

Step	Action
1	Load every telemetry CSV into one table and keep node_id and source job ID as metadata.
2	Load the matching stress_active CSV and read stress_start and stress_end.
3	Create the target: label 2 inside the interval, label 0 outside.
4	Sort by job, node, and timestamp.
5	Fill missing delta values only when they are the expected first-row counter reset; do not silently turn errors into zero.
6	Normalize or standardize numeric features using training data only.

Step	Action
7	Split by complete job or experiment run, not by random individual rows.

> Why split by job? Adjacent 0.5-second samples are highly correlated. A random row split can place nearly identical neighboring samples in both training and test sets and make accuracy look better than it really is.

6. Minimal starter training recipe

A teammate can start with a binary classifier: normal versus Label 2. Logistic regression gives an interpretable baseline. A tree-based model can be tested next if nonlinear relationships matter.

```
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

FEATURES = [
    "gpu_util_pct", "gpu_mem_used_mb",
    "ib_port_xmit_data_delta", "ib_port_rcv_data_delta",
    "ib_port_xmit_packets_delta", "ib_port_rcv_packets_delta",
    "ib_port_xmit_wait_delta",
    "ib_port_xmit_discards_delta",
    "ib_sq_num_rnr_delta", "ib_sq_num_to_delta",
]

# df must already contain label and a run_id/job_id group column
X = df[FEATURES]
y = (df["label"] == 2).astype(int)
groups = df["run_id"]

splitter = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=42)
train_idx, test_idx = next(splitter.split(X, y, groups=groups))

model = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    LogisticRegression(max_iter=2000, class_weight="balanced"),
)
model.fit(X.iloc[train_idx], y.iloc[train_idx])
pred = model.predict(X.iloc[test_idx])
print(confusion_matrix(y.iloc[test_idx], pred))
print(classification_report(y.iloc[test_idx], pred, digits=3))
```

For the first evaluation, report precision, recall, F1 score, and the confusion matrix. Recall is especially important if missing a real Label 2 event is costly. Also report false alarms on healthy Label 0 data.

7. Do not use these fields as predictive features

Field	Reason
stress_active	It directly reveals the target interval; use it only to create labels.
tool	It says ib_write_bw was active and would leak the experiment setup.
job_id	An identifier, not a physical signal.
hostname / node_id	Use for grouping and generalization checks, not as the main signal.
timestamp	Use for ordering and joining, not as a raw model feature.
cumulative counters	They mostly represent how long the job has been running; prefer deltas or rates.

Important limitation

The current Label 2 dataset is a controlled contention dataset. It demonstrates that extra RDMA traffic changes InfiniBand counters and can affect distributed training, but it does not represent every real production failure. The model should be

described as recognizing the collected Label 2 pattern, not as proving that every possible network problem is Label 2.

The supplied RDMA logs show a 64 KB ib_write_bw workload with approximately 4,679.90 MiB/s average bandwidth in the example run. This is useful run metadata, not a per-row feature.

8. Handoff checklist

Required	Status to verify
Telemetry CSVs	At least one telemetry file per node and run.
Stress-marker CSVs	Matching start/end interval for every Label 2 run.
Run metadata	Node pair, parameter size, stress duration, and job ID.
Target label	0 outside the interval and 2 inside the interval.
Feature table	Deltas selected, missing values handled, optional step time joined.
Evaluation split	Train/test separated by complete job or run.
Evaluation report	Precision, recall, F1, confusion matrix, and false-alarm rate.

Recommended first milestone

Build one merged table from the supplied run, train a baseline classifier, and verify that Label 2 rows show higher transmit-wait and traffic features than Label 0 rows. Then add more independent jobs and node pairs before drawing conclusions about generalization.

Source files used for this handoff: telemetry_g8.hpc.coe_105175.csv, telemetry_g11.hpc.coe_105175.csv, stress_active_g8.hpc.coe.csv, stress_active_g11.hpc.coe.csv, node_0_0(1).log, node_1_1(1).log, rdma_stress_g8.hpc.coe.log, and rdma_stress_g11.hpc.coe.log.